

構文解析

triC 2026/06/20

今日やること

- 構文解析でやりたいこと、構文解析のやりかた
- BNFについて、四則演算のBNF
- BNFから再帰関数にする方法
- 構文解析の動き
- 演習問題

構文解析でやりたいこと

- $4 + 8 * (3 + 7) =$ とか文字列が与えられ、それを評価(計算)する
- BNF記法で生成できる文字列が与えられ、それを木構造に変換(その後問題を解く)

$\langle \text{expression} \rangle ::= \langle \text{number} \rangle \mid \text{if_} \langle \text{variable} \rangle _ \text{then_} \langle \text{expression} \rangle _ \text{else_} \langle \text{expression} \rangle$

$\langle \text{number} \rangle ::= 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

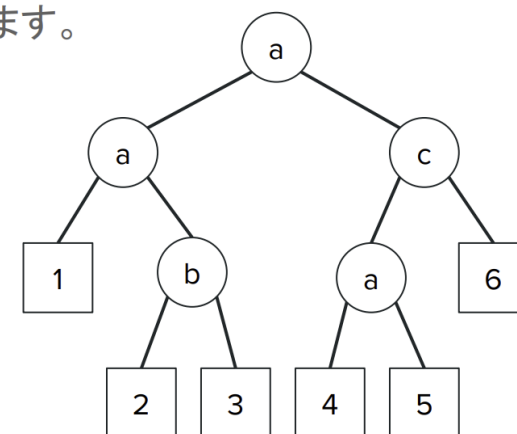
$\langle \text{variable} \rangle ::= \langle \text{letter} \rangle \mid \langle \text{letter} \rangle \langle \text{variable} \rangle$

$\langle \text{letter} \rangle ::= a \mid b \mid c \mid d \mid e \mid f \mid g \mid h \mid i \mid j \mid k \mid l \mid m \mid$
 $n \mid o \mid p \mid q \mid r \mid s \mid t \mid u \mid v \mid w \mid x \mid y \mid z$

BNFの例

構文解析後、右のような木を構築することができます。

- この例は
if_a_then_if_a_then_1_else_if_b_then_2_else_3_
else_if_c_then_if_a_then_4_else_5_else_6
に相当。



構文解析のやり方

1. 処理をする文字列を生成するBNFを作成(与えられることがほとんど)
2. BNFの各式に対応する再帰関数を書く

BNFについて

- これを見る！
- 再帰関数との対応を意識して見る

<https://medium-company.com/bnf記法-基本情報技術者試験/>

その他(正規表現)

- * :0回以上の連続 A* なら $\emptyset$,A,AAAとか
- + :1回以上の連続 A+ならA,AA,AAAとか

四則演算のBNF(1/2)

- 四則演算のBNF

expr ::= **term** [(**+** | **-**) **term**]*
term ::= **factor** [(***** | **/**) **factor**]*
factor ::= **number** | '(' **expr** ')'
number ::= 1桁以上の数字

- **expr** : +や-で**term**をつなげた式 例) **term** + **term** - **term**
- **term** : *や/で**factor**をつなげた式 例) **factor** * **factor** / **factor**
- **factor** : 数値や括弧. 括弧の中は**expr**. 例) **number** , (**expr**)
- **number** : 1桁以上の数字 例) 1, 23 , 100

四則演算のBNF(2/2)

expr ::= **term** [('+' | '-') **term**]*
term ::= **factor** [('*' | '/') **factor**]*
factor ::= **number** | '(' **expr** ')'
number ::= 1桁以上の数字

- 四則演算が上記のルールに即しているかの確認

例) 2*(3+4)+5

=2*(3+4)+5

=2*(3+4)+5

=2*(3+4)+5

=2*(3+4)+5

=2*(3+4)+5

=2*(3+4)+5

=2*(3+4)+5

BNFから再帰関数へ(1/3)

- どうやって式が与えられた時に解くのか
-> BNFと対応する再帰関数を作成し、左から1文字ずつ読んで計算する

- expr

expr ::= term [('+' | '-') term]*

```
int expr(string& s, int& i) {  
    int val = term(s, i);  
    while(s[i] == '+' || s[i] == '-') {  
        char op = s[i];  
        i++;  
        int val2 = term(s, i);  
        if (op == '+') val += val2;  
        else val -= val2;  
    }  
    return val;  
}
```


BNFから再帰関数へ(2/3)

- term

`term ::= factor [('*' | '/') factor]*`

```
int term(string& s, int& i) {
    int val = factor(s, i);
    while(s[i] == '*' || s[i] == '/') {
        char op = s[i];
        i++;
        int val2 = factor(s, i);
        if (op == '*') val *= val2;
        else val /= val2;
    }
    return val;
}
```

- factor

`factor ::= number | '(' expr ')'`

```
int factor(string& s, int& i) {
    if (isdigit(s[i])) return number(s, i);

    // ここで構文が正しければ s[i] == '('
    i++; // '('を読み飛ばす
    int ret = expr(s, i);
    i++; // ')'を読み飛ばす
    return ret;
}
```

BNFから再帰関数へ(3/3)

- number

2桁以上にも対応できる実装

```
int number(string& s, int& i) {  
    int n = s[i++] - '0';  
    while(isdigit(s[i])) n = n*10 + s[i++] - '0';  
    return n;  
}
```

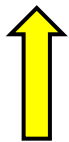
- main関数からexpr()を呼べばいい
- 自分より下の関数を呼ぶのでプロトタイプ宣言を忘れずに

```
int main() {  
    string str = "2*(3+4)+5=";  
    int i = 0;  
    cout << str << expr(str, i) << endl; // => 19  
    return 0;  
}
```

構文解析の動き

```
expr ::= term [( '+' | '-' ) term]*  
term  ::= factor [( '*' | '/' ) factor]*  
factor ::= number | '(' expr ')'  
number ::= 1桁以上の数字
```

2 * (3 + 4) + 5



main()関数からexpr()が呼ばれる



```
int main() {  
    string str = "2*(3+4)+5=";  
    int i = 0;  
    cout << str << expr(str, i) << endl; // => 19  
    return 0;  
}
```

構文解析の動き

```
expr ::= term [( '+' | '-' ) term]*  
term  ::= factor [( '*' | '/' ) factor]*  
factor ::= number | '(' expr ')'  
number ::= 1桁以上の数字
```

2 * (3 + 4) + 5



expr()

→ term()

→ factor()

→ number() で2を読んで返す

構文解析の動き

```
expr    ::= term [( '+' | '-' ) term]*  
term    ::= factor [( '*' | '/' ) factor]*  
factor  ::= number | '(' expr ')'  
number  ::= 1桁以上の数字
```

2 * (3 + 4) + 5



expr()

→ term()が'+'を見つける、右のfactorを読みに行く

構文解析の動き

```
expr ::= term [( '+' | '-' ) term]*  
term  ::= factor [( '*' | '/' ) factor]*  
factor ::= number | '(' expr ')'  
number ::= 1桁以上の数字
```

2 * (3 + 4) + 5



expr()

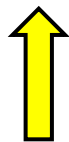
→ term()

→ factor() が '('を確認、中身のexprを読みに行く

構文解析の動き

```
expr ::= term [( '+' | '-' ) term]*  
term  ::= factor [( '*' | '/' ) factor]*  
factor ::= number | '(' expr ')'  
number ::= 1桁以上の数字
```

2 * (3 + 4) + 5



expr()

→ term()

→ factor()

→ expr()

→ term()

→ factor()

→ number()で3を読んで返す

構文解析の動き

```
expr    ::= term [( '+' | '-' ) term]*  
term    ::= factor [( '*' | '/' ) factor]*  
factor  ::= number | '(' expr ')'  
number  ::= 1桁以上の数字
```

2 * (3 + 4) + 5



expr()

→ term()

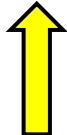
→ factor()

→ expr()が '+' を見つける、右のtermを読みに行く

構文解析の動き

```
expr ::= term [( '+' | '-' ) term]*  
term  ::= factor [( '*' | '/' ) factor]*  
factor ::= number | '(' expr ')'  
number ::= 1桁以上の数字
```

2 * (3 + 4) + 5



expr()

→ term()

→ factor()

→ expr()

→ term()

→ factor()

→ number()で4を読んで返す

構文解析の動き

2 * (3 + 4) + 5



expr()

→ term()

→ factor()

→ expr()で3+4=7が計算されて返される

```
expr    ::= term [( '+' | '-' ) term]*  
term    ::= factor [( '*' | '/' ) factor]*  
factor  ::= number | '(' expr ')'  
number  ::= 1桁以上の数字
```

```
int expr(string& s, int& i) {  
    int val = term(s, i);  
    while(s[i] == '+' || s[i] == '-') {  
        char op = s[i];  
        i++;  
        int val2 = term(s, i);  
        if (op == '+') val += val2;  
        else val -= val2;  
    }  
    return val;  
}
```

構文解析の動き

2 * (3 + 4) + 5



expr()

→ term()

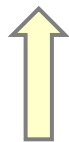
→ factor()で')'を確認、(3+4)の計算結果7が返される

```
expr    ::= term [( '+' | '-' ) term]*  
term    ::= factor [( '*' | '/' ) factor]*  
factor  ::= number | '(' expr ')'  
number  ::= 1桁以上の数字
```

```
int factor(string& s, int& i) {  
    if (isdigit(s[i])) return number(s, i);  
  
    // ここで構文が正しければ s[i] == '('  
    i++; // '('を読み飛ばす  
    int ret = expr(s, i);  
    i++; // ')'を読み飛ばす  
    return ret;  
}
```

構文解析の動き

2 * (3 + 4) + 5



expr()

→ term()で2*7=14が計算され返される

```
expr    ::= term [( '+' | '-' ) term]*  
term    ::= factor [( '*' | '/' ) factor]*  
factor  ::= number | '(' expr ')'  
number  ::= 1桁以上の数字
```

```
int term(string& s, int& i) {  
    int val = factor(s, i);  
    while(s[i] == '*' || s[i] == '/') {  
        char op = s[i];  
        i++;  
        int val2 = factor(s, i);  
        if (op == '*') val *= val2;  
        else val /= val2;  
    }  
    return val;  
}
```

構文解析の動き

```
expr    ::= term [( '+' | '-' ) term]*  
term    ::= factor [( '*' | '/' ) factor]*  
factor  ::= number | '(' expr ')'  
number  ::= 1桁以上の数字
```

2 * (3 + 4) + 5



expr()が '+' を発見し、右のtermを読みに行く

構文解析の動き

```
expr    ::= term [( '+' | '-' ) term]*  
term    ::= factor [( '*' | '/' ) factor]*  
factor  ::= number | '(' expr ')'  
number  ::= 1桁以上の数字
```

2 * (3 + 4) + 5
 ↑

expr()

→ term()

→ factor()

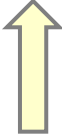
→ number() で5を読んで返す

構文解析の動き

2 * (3 + 4) + 5

expr()が14+5=19を返す

```
expr    ::= term [( '+' | '-' ) term]*  
term    ::= factor [( '*' | '/' ) factor]*  
factor  ::= number | '(' expr ')'  
number  ::= 1桁以上の数字
```



```
int expr(string& s, int& i) {  
    int val = term(s, i);  
    while(s[i] == '+' || s[i] == '-') {  
        char op = s[i];  
        i++;  
        int val2 = term(s, i);  
        if (op == '+') val += val2;  
        else val -= val2;  
    }  
    return val;  
}
```

構文解析の動き

```
expr    ::= term [( '+' | '-' ) term]*  
term    ::= factor [( '*' | '/' ) factor]*  
factor  ::= number | '(' expr ')'  
number  ::= 1桁以上の数字
```

2 * (3 + 4) + 5

main()関数に戻ってきてexpr()から返された19を出力。終わり



```
int main() {  
    string str = "2*(3+4)+5=";  
    int i = 0;  
    cout << str << expr(str, i) << endl; // => 19  
    return 0;  
}
```


演習問題

<https://onlinejudge.u-aizu.ac.jp/challenges/sources/JAG/Prelim/2883>

方針:

1. a, b, c, d の値を全探索する
2. その a, b, c, d の値に対して式 S を構文解析しながら計算する
3. 条件を満たすか判定する

その他

- 構文木にするには？

計算結果ではなく、ノードを返して、つなげて木構造にする

- 参考になるサイト

<https://gist.github.com/draftcode/1357281>

古いけどよく使われる。イテレータを管理する実装。

<https://blog.hamayanhamayan.com/entry/2018/07/13/085956>

<https://dai1741.github.io/maximum-algo-2012/docs/parsing/>